Deliverable 3 - INF201: Battle Simulator

Group members: *Name 1*, *Name 2*

GitHub repository: *(add link here)*

Date: 13 November 2026

In this deliverable we extend the `Character` class from lecture 08 (`game_simulator_v1.py`) into a small turn-based battle simulator. Two characters fight until one of them dies. Damage comes either from a fixed base damage or from a weapon whose damage is decided by a dice roll, and characters can specialise as a **Warrior** (armor) or a **Mage** (mana and spells).

Project structure

File	Contents
`dice.py`	`Dice` - an n-sided die (n >= 2)
`items.py`	`Weapon` - a named weapon that rolls a `Dice` for damage
`character.py`	`Character` base class and the subclasses `Warrior` and `Mage`
`battle.py`	`Battle` - runs turn-based combat between two characters
`test_game.py`	`pytest` tests for all classes (`python -m pytest`)
`game_simulator_v1.py`	The original starting code from lecture 08 (unchanged, for reference)
`GENAI_USE.md`	Discussion of how we used generative AI

The notebook first presents each class (attributes and methods, with small demos), and then runs simulated battles between different types of characters, with and without weapons.

In [1]:
```
import inspect
import random
from collections import Counter

from IPython.display import Markdown, display

from battle import Battle
from character import Character, Mage, Warrior
from dice import Dice
from items import Weapon

random.seed(201)  # fixed seed so the random results in this notebook are reproducible
```

We use a small helper function to present a class: it lists the constructor, the properties (and whether they are read-only), and the methods, together with the first line of their docstrings.

For subclasses it also shows which methods are **new or overridden** in the subclass and which are **inherited**.

In [2]:
```
SHOWN_DUNDERS = {"__repr__", "__str__", "__call__"}


def describe_class(cls) -> None:
    """Print the constructor, class constants, properties and public methods of a class."""
    bases = ", ".join(b.__name__ for b in cls.__bases__)
    print(f"class {cls.__name__}({bases})")
    print(f"    {inspect.getdoc(cls).splitlines()[0]}")
    print(f"\n  Constructor: {cls.__name__}{inspect.signature(cls)}")

    constants = [n for n in vars(cls) if n.isupper()]
    if constants:
        print("\n  Class constants:")
        for name in constants:
            print(f"    {name} = {getattr(cls, name)}")

    properties, methods = [], []
    for name, member in inspect.getmembers(cls):
        if name.startswith("_") and name not in SHOWN_DUNDERS:
            continue
        if not (isinstance(member, property) or inspect.isfunction(member)):
            continue
        defined_here = name in vars(cls) or cls.__bases__ == (object,)
        where = "" if defined_here else "  [inherited]"
        doc = (inspect.getdoc(member) or "").splitlines()[0]
        if isinstance(member, property):
            access = "read-only" if member.fset is None else "read/write"
            properties.append(f"    {name:<24} ({access}) {doc}{where}")
        else:
            signature = str(inspect.signature(member)).replace("(self, ", "(").replace("(self)",
"()")
            methods.append(f"    {name}{signature}\n        {doc}{where}")

    print("\n  Properties:")
    print("\n".join(properties) if properties else "    (none - name and damage_dice are plain
attributes)")
    print("\n  Methods:")
    print("\n".join(methods))
```

Task 1: Updating the `Character` class

In the original code, `Character.attack()` changed the *target's* protected attribute `_hp` directly. That breaks encapsulation: an object should only change its own private state. We fixed this by:

- adding `take_damage(amount)`, which only changes the character's **own** `_hp` and never lets it go below 0,
- adding `heal(amount)`, which increases `_hp` but never above `hp_max` (a dead character cannot be healed),
- changing `attack(target)` so it calls `target.take_damage(...)` instead of touching `target._hp`,
- adding a read-only `is_alive` property that returns `True` if `_hp > 0`.

Other design choices:

- `hp`, `hp_max`, `damage` and `is_alive` are **read-only properties**, so the rule *0 <= HP <= hp_max* can only be broken by going around the public interface. `hp_max` is read-only so it can never be set below the current HP.
- Inputs are validated: for example, negative damage or healing raises a `ValueError` (otherwise `take_damage(-50)` would secretly heal the character).
- `take_damage()`, `heal()` and `attack()` **return** how much HP actually changed. This is useful in the battle log and in the tests.
- `__repr__` uses `type(self).__name__`, so subclasses automatically show the right class name (we also fixed a stray `)` in the original `__repr__`).

In [3]:
```
describe_class(Character)
```

```
class Character(object)
    Represents a character in the RPG.

  Constructor: Character(name: str, hp_max: int = 100, damage: int = 10, weapon: items.Weapon |
None = None) -> None

  Properties:
    damage                  (read-only) Read-only base damage, used when the character has no
weapon equipped.
    hp                      (read-only) Public read-only property to inspect HP without allowing
direct, unsafe edits.
    hp_max                  (read-only) Read-only max HP (read-only so HP can never end up above
the max).
    is_alive                (read-only) Read-only: True if the character has more than 0 HP, False
otherwise.
    status                  (read-only) Short status string used in battle logs, e.g. ``'84/100
HP'``.
    weapon                  (read/write) The equipped weapon, or None if the character fights with
bare hands.

  Methods:
    __repr__() -> str
        Representation of character object.
    __str__() -> str
        String representation of character object.
    attack(target: 'Character') -> int
        Attack the target character.
    heal(amount: int) -> int
        Increase the character's HP by ``amount``, never going above ``hp_max``.
    introduce() -> None
        Print a line that introduces the character.
    next_attack_description() -> str
        Describe what the next attack will be, used in battle logs.
    take_damage(amount: int) -> int
        Reduce the character's *own* HP by ``amount``, never going below 0.
```

In [4]:
```
arthur = Character("Arthur")
lancelot = Character("Lancelot", hp_max=80, damage=15)

arthur.introduce()
print(repr(arthur))
print(lancelot)
print("Private attributes of the object:", vars(lancelot))
```

```
Hi, my name is Arthur! I am a character.
Character(name='Arthur', hp=100, hp_max=100, damage=10, weapon=None)
Character Lancelot: 80/80 HP, base damage 15, weapon: none
Private attributes of the object: {'name': 'Lancelot', '_hp_max': 80, '_hp': 80, '_damage': 15,
'_weapon': None}
```

In [5]:
```python
# attack() calls the target's take_damage() method and returns the damage dealt
dealt = arthur.attack(lancelot)
print(f"Arthur deals {dealt} damage -> Lancelot has {lancelot.hp}/{lancelot.hp_max} HP")

# take_damage() never lets HP go below 0
lost = lancelot.take_damage(500)
print(f"Lancelot takes 500 damage but only loses {lost} HP -> hp={lancelot.hp}, is_alive=
{lancelot.is_alive}")

# heal() never goes above hp_max, and a dead character cannot be healed
arthur.take_damage(30)
print(f"Arthur heals {arthur.heal(50)} HP -> {arthur.hp}/{arthur.hp_max} HP")
print(f"Lancelot (dead) heals {lancelot.heal(50)} HP -> {lancelot.hp}/{lancelot.hp_max} HP")
```

```
Arthur deals 10 damage -> Lancelot has 70/80 HP
Lancelot takes 500 damage but only loses 70 HP -> hp=0, is_alive=False
Arthur heals 30 HP -> 100/100 HP
Lancelot (dead) heals 0 HP -> 0/80 HP
```

In [6]:
```python
# hp and is_alive are read-only, and invalid input is rejected
for action in (lambda: setattr(arthur, "hp", 1000),
               lambda: setattr(arthur, "is_alive", True),
               lambda: arthur.take_damage(-50),
               lambda: arthur.attack(arthur)):
    try:
        action()
    except (AttributeError, ValueError) as error:
        print(f"{type(error).__name__}: {error}")
```

```
AttributeError: property 'hp' of 'Character' object has no setter
AttributeError: property 'is_alive' of 'Character' object has no setter
ValueError: amount must be at least 0, got -50
ValueError: A character cannot attack itself
```

Task 2: Randomness with the `Dice` class

`Dice` models an n-sided die. The number of sides is stored in a private attribute `_sides` and exposed through a read-only property, because a die should not change shape after it is made. Creating a die with fewer than 2 sides raises a `ValueError`, and a non-integer number of sides raises a `TypeError`.

- `roll()` returns `random.randint(1, self.sides)`.
- `__repr__` returns e.g. `Dice(sides=6)`.
- `__call__` makes the object callable, so `d20()` does the same as `d20.roll()`.

In [7]:
```python
describe_class(Dice)
```

```
class Dice(object)
    An n-sided die that can be rolled to produce a random integer from 1 to n.

  Constructor: Dice(sides: int = 6) -> None

  Properties:
    sides                    (read-only) Read-only number of sides on the die.

  Methods:
    __call__() -> int
        Make the die callable, so ``d20()`` is the same as ``d20.roll()``.
    __repr__() -> str
        Representation of the die, e.g. ``Dice(sides=6)``.
    roll() -> int
        Roll the die and return a random integer between 1 and ``sides`` (inclusive).
```

In [8]:
```python
d6 = Dice()
d20 = Dice(sides=20)
print(d6, d20)
print("d20.roll():", d20.roll())
print("d20():     ", d20(), "  <- __call__ makes the die callable")

for bad_sides in (1, 0, "six"):
    try:
        Dice(bad_sides)
    except (TypeError, ValueError) as error:
        print(f"Dice({bad_sides!r}) -> {type(error).__name__}: {error}")
```

```
Dice(sides=6) Dice(sides=20)
d20.roll(): 3
d20():      11   <- __call__ makes the die callable
Dice(1) -> ValueError: A dice must have at least 2 sides, got 1
Dice(0) -> ValueError: A dice must have at least 2 sides, got 0
Dice('six') -> TypeError: sides must be an int, got str
```

To check that the die is fair, we roll a d6 60 000 times. Each face should come up about 10 000 times (1/6 of the rolls).

In [9]:
```python
counts = Counter(d6() for _ in range(60_000))
for face in sorted(counts):
    print(f"{face}: {counts[face]:>6}  {'#' * (counts[face] // 250)}")
```

```
1:  10166  ########################################
2:   9999  ########################################
3:   9972  ########################################
4:   9956  ########################################
5:   9903  ########################################
6:  10004  ########################################
```

Task 3: Items - the `Weapon` class

`Weapon` (in `items.py`) has a `name` and a `damage_dice: Dice`. This is **composition**: a weapon *has a* die, and a character *has a* weapon. `roll_damage()` simply rolls the weapon's die.

`Character.__init__` now accepts an optional `weapon` (default `None`). The `weapon` is a property with a setter that checks the value is a `Weapon` (or `None`), so a character can change weapons during the game but cannot be given e.g. a string. If a weapon is equipped, `attack()` rolls the weapon's damage instead of using the base damage.

Note that the weapon *replaces* the base damage. A weapon whose die has *n* sides deals on average (n+1)/2 damage, so a weapon only helps if this is larger than the character's base damage (10 by default). A `Dagger` (d4, mean 2.5) is therefore worse than bare hands, while a `Greatsword` (d20, mean 10.5) is slightly better on average, but much more random.

In [10]:
```
describe_class(Weapon)
```

```
class Weapon(object)
    A weapon whose damage is decided by rolling its damage dice.

  Constructor: Weapon(name: str, damage_dice: dice.Dice) -> None

  Properties:
    (none - name and damage_dice are plain attributes)

  Methods:
    __repr__() -> str
        Representation of the weapon, e.g. ``Weapon(name='Dagger', damage_dice=Dice(sides=4))``.
    __str__() -> str
        Readable description of the weapon, e.g. ``Dagger (1d4)``.
    roll_damage() -> int
        Roll the weapon's damage dice and return the damage for one attack.
```

In [11]:
```
dagger = Weapon("Dagger", Dice(4))
staff = Weapon("Staff", Dice(6))
longsword = Weapon("Longsword", Dice(12))
greatsword = Weapon("Greatsword", Dice(20))

for weapon in (dagger, staff, longsword, greatsword):
    rolls = [weapon.roll_damage() for _ in range(10)]
    mean = (weapon.damage_dice.sides + 1) / 2
    print(f"{str(weapon):<18} mean {mean:>4}, 10 rolls: {rolls}")
print(repr(greatsword))
```

```
Dagger (1d4)       mean  2.5, 10 rolls: [4, 3, 3, 3, 2, 4, 2, 1, 4, 2]
Staff (1d6)        mean  3.5, 10 rolls: [2, 3, 6, 5, 1, 6, 3, 6, 2, 1]
Longsword (1d12)   mean  6.5, 10 rolls: [3, 7, 7, 12, 12, 6, 9, 1, 1, 6]
Greatsword (1d20)  mean 10.5, 10 rolls: [3, 5, 11, 3, 16, 2, 5, 5, 12, 14]
Weapon(name='Greatsword', damage_dice=Dice(sides=20))
```

In [12]:
```
# A character with a weapon rolls the weapon's damage instead of using base damage
robin = Character("Robin", weapon=longsword)
target = Character("Training dummy", hp_max=1000)
print(robin)
print("Damage of 8 attacks:", [robin.attack(target) for _ in range(8)])

# Weapons can be changed (or removed) through the validated property
robin.weapon = None
print("Bare hands, 3 attacks:", [robin.attack(target) for _ in range(3)])
try:
    robin.weapon = "a big stick"
except TypeError as error:
    print(f"TypeError: {error}")
```

```
Character Robin: 100/100 HP, base damage 10, weapon: Longsword (1d12)
Damage of 8 attacks: [7, 8, 4, 10, 5, 10, 1, 6]
Bare hands, 3 attacks: [10, 10, 10]
TypeError: weapon must be a Weapon or None, got str
```

Task 4: Character specialisation - `Warrior` and `Mage`

Both classes inherit from `Character` and call `super().__init__(...)`, so they get all the attributes and methods of a character, and only add or override what is special for them.

`Warrior` (defensive specialist)

- New read-only attribute `armor` (default 5).
- Overrides `take_damage()`: incoming damage is reduced by `armor` (but never below 0, so weak attacks cannot heal the warrior) before calling `Character.take_damage()` through `super()`.

`Mage` (resource-based attacker)

- New read-only attributes `mana` and `mana_max` (both default 30).
- Overrides `attack()`:
  - With at least 10 mana: spend 10 mana on a spell that deals **double base damage**. The spell ignores any weapon.
  - With less than 10 mana: do the normal `Character.attack()` (using the weapon if equipped) and restore 5 mana, never above `mana_max`.
- The numbers 10, 5 and 2 are class constants (`SPELL_COST`, `MANA_REGEN`, `SPELL_MULTIPLIER`) instead of "magic numbers" in the code.

Both subclasses also extend `__repr__` / `__str__` and the short `status` used in the battle log. `Mage` overrides `next_attack_description()` so the battle log can say whether the next attack is a spell.

In [13]:
```
describe_class(Warrior)
```

```
class Warrior(Character)
    Defensive specialist: armor reduces all incoming damage.

  Constructor: Warrior(name: str, hp_max: int = 100, damage: int = 10, weapon: items.Weapon | None
= None, armor: int = 5) -> None

  Properties:
    armor                   (read-only) Read-only armor value.
    damage                  (read-only) Read-only base damage, used when the character has no
weapon equipped.  [inherited]
    hp                      (read-only) Public read-only property to inspect HP without allowing
direct, unsafe edits.  [inherited]
    hp_max                  (read-only) Read-only max HP (read-only so HP can never end up above
the max).  [inherited]
    is_alive                (read-only) Read-only: True if the character has more than 0 HP, False
otherwise.  [inherited]
    status                  (read-only) Short status string used in battle logs, e.g. ``'84/100
HP'``.  [inherited]
    weapon                  (read/write) The equipped weapon, or None if the character fights with
bare hands.  [inherited]

  Methods:
    __repr__() -> str
        Representation of character object.  [inherited]
    __str__() -> str
        String representation of character object.  [inherited]
    attack(target: 'Character') -> int
        Attack the target character.  [inherited]
    heal(amount: int) -> int
        Increase the character's HP by ``amount``, never going above ``hp_max``.  [inherited]
    introduce() -> None
        Print a line that introduces the character.  [inherited]
    next_attack_description() -> str
        Describe what the next attack will be, used in battle logs.  [inherited]
    take_damage(amount: int) -> int
        Reduce incoming damage by ``armor`` (not below 0) before applying it.
```

In [14]:

```python
conan = Warrior("Conan")
print(repr(conan))
print("Warrior is a subclass of Character:", issubclass(Warrior, Character))
print("Method resolution order:", [c.__name__ for c in Warrior.__mro__])

for hit in (12, 5, 3):
    lost = conan.take_damage(hit)
    print(f"Incoming {hit:>2} damage - armor {conan.armor} -> loses {lost} HP, now
{conan.status}")
```

```
Warrior(name='Conan', hp=100, hp_max=100, damage=10, weapon=None, armor=5)
Warrior is a subclass of Character: True
Method resolution order: ['Warrior', 'Character', 'object']
Incoming 12 damage - armor 5 -> loses 7 HP, now 93/100 HP
Incoming  5 damage - armor 5 -> loses 0 HP, now 93/100 HP
Incoming  3 damage - armor 5 -> loses 0 HP, now 93/100 HP
```

In [15]:

```python
describe_class(Mage)
```

```
class Mage(Character)
    Resource-based attacker: spends mana on spells that deal double damage.

  Constructor: Mage(name: str, hp_max: int = 100, damage: int = 10, weapon: items.Weapon | None =
None, mana: int = 30, mana_max: int = 30) -> None

  Class constants:
    SPELL_COST = 10
    MANA_REGEN = 5
    SPELL_MULTIPLIER = 2

  Properties:
    damage                  (read-only) Read-only base damage, used when the character has no
weapon equipped.  [inherited]
    hp                      (read-only) Public read-only property to inspect HP without allowing
direct, unsafe edits.  [inherited]
    hp_max                  (read-only) Read-only max HP (read-only so HP can never end up above
the max).  [inherited]
    is_alive                (read-only) Read-only: True if the character has more than 0 HP, False
otherwise.  [inherited]
    mana                    (read-only) Read-only current mana.
    mana_max                (read-only) Read-only max mana.
    status                  (read-only) Short status string including mana, e.g. ``'84/100 HP,
20/30 mana'``.
    weapon                  (read/write) The equipped weapon, or None if the character fights with
bare hands.  [inherited]

  Methods:
    __repr__() -> str
        Representation of character object.  [inherited]
    __str__() -> str
        String representation of character object.  [inherited]
    attack(target: character.Character) -> int
        Attack the target, using a spell if there is enough mana.
    can_cast_spell() -> bool
        True if the mage has enough mana to cast a spell.
    heal(amount: int) -> int
        Increase the character's HP by ``amount``, never going above ``hp_max``.  [inherited]
    introduce() -> None
        Print a line that introduces the character.  [inherited]
    next_attack_description() -> str
        Describe what the next attack will be, used in battle logs.
    take_damage(amount: int) -> int
        Reduce the character's *own* HP by ``amount``, never going below 0.  [inherited]
```

In [16]:

```python
merlin = Mage("Merlin", weapon=staff)
dummy = Character("Training dummy", hp_max=1000)
print(repr(merlin))
print()
for turn in range(1, 10):
    description = merlin.next_attack_description()
    dealt = merlin.attack(dummy)
    print(f"Attack {turn}: {description:<36} -> {dealt:>2} damage, mana now
{merlin.mana:>2}/{merlin.mana_max}")
```

```
Mage(name='Merlin', hp=100, hp_max=100, damage=10, weapon=Weapon(name='Staff',
damage_dice=Dice(sides=6)), mana=30, mana_max=30)

Attack 1: a magic spell (20 dmg, 10 mana)      -> 20 damage, mana now 20/30
Attack 2: a magic spell (20 dmg, 10 mana)      -> 20 damage, mana now 10/30
Attack 3: a magic spell (20 dmg, 10 mana)      -> 20 damage, mana now  0/30
Attack 4: Staff (1d6)                          ->  4 damage, mana now  5/30
Attack 5: Staff (1d6)                          ->  2 damage, mana now 10/30
Attack 6: a magic spell (20 dmg, 10 mana)      -> 20 damage, mana now  0/30
Attack 7: Staff (1d6)                          ->  6 damage, mana now  5/30
Attack 8: Staff (1d6)                          ->  4 damage, mana now 10/30
Attack 9: a magic spell (20 dmg, 10 mana)      -> 20 damage, mana now  0/30
```

The mage starts with 30 mana, so it casts three spells (20 damage each) in a row. After that, mana drops to 0 and the pattern becomes *weapon, weapon, spell*: two normal attacks restore 5 + 5 = 10 mana, which is exactly enough for one more spell. The normal attacks use the equipped `Staff` (d6), while the spells ignore it.

In [17]:
```python
# A spell against a warrior is reduced by the warrior's armor (20 - 5 = 15)
conan = Warrior("Conan")
dealt = Mage("Merlin").attack(conan)
print(f"Spell deals {dealt} damage to the warrior -> {conan.status}")

try:
    Mage("Broken", mana=40, mana_max=30)
except ValueError as error:
    print(f"ValueError: {error}")
```

```
Spell deals 15 damage to the warrior -> 85/100 HP
ValueError: mana (40) cannot be larger than mana_max (30)
```

Task 5: Running battles with the `Battle` class

`Battle` is initialised with a `hero` and an `enemy` (two different `Character` objects).

- `play_round()`: the hero attacks the enemy, and if the enemy is still alive, the enemy attacks the hero. Calling it after the battle is over raises a `RuntimeError`.
- `simulate_battle()`: uses a `while` loop to play rounds until one character is no longer alive, and returns the winner.
- Read-only properties `winner`, `is_over` and `rounds_played` describe the state of the battle.

Two extra design choices:

- **`verbose`**: when `True`, every attack is printed. When running thousands of battles for statistics we set it to `False`.
- **`max_rounds`** (default 1000): if neither character can hurt the other (for example two warriors whose base damage is lower than the other's armor), the `while` loop would never end. After `max_rounds` rounds the battle ends in a draw and `simulate_battle()` returns `None`.

Because `Battle` only uses the public interface of `Character` (`attack()`, `is_alive`, `status`, ...), it works for any subclass - this is **polymorphism**: the same code makes a

`Mage` cast spells and a `Warrior` block damage with armor.

In [18]:
```
describe_class(Battle)
```

```
class Battle(object)
    Turn-based combat between a hero and an enemy, fought until one of them dies.

  Constructor: Battle(hero: character.Character, enemy: character.Character, verbose: bool = True,
max_rounds: int = 1000) -> None

  Properties:
    enemy                   (read-only) The character that attacks second (read-only).
    hero                    (read-only) The character that attacks first (read-only).
    is_over                 (read-only) True when at least one of the characters is no longer
alive.
    rounds_played           (read-only) Number of rounds played so far (read-only).
    winner                  (read-only) The surviving character, or None if the battle is not
decided.

  Methods:
    play_round() -> None
        Play one round: the hero attacks the enemy, and if the enemy is still alive,
    simulate_battle() -> character.Character | None
        Play rounds until one character is no longer alive.
```

Playing single rounds

We first play two rounds by hand with `play_round()`.

In [19]:
```
hero = Character("Arthur")
enemy = Character("Mordred", damage=8)
battle = Battle(hero, enemy)
battle.play_round()
battle.play_round()
print(f"Rounds played: {battle.rounds_played}, battle over: {battle.is_over}, winner:
{battle.winner}")
```

```
Round 1
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 92/100 HP | Mordred: 90/100 HP
Round 2
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 84/100 HP | Mordred: 80/100 HP
Rounds played: 2, battle over: False, winner: None
```

Battle 1: Character vs Character, no weapons

Without weapons there is no randomness, so this battle always plays out the same way. Arthur deals 10 damage per round and Mordred deals 8, and both have 100 HP. Arthur needs 10 rounds to win, and because he attacks first, Mordred never gets his 10th attack.

In [20]:
```
battle = Battle(Character("Arthur"), Character("Mordred", damage=8))
winner = battle.simulate_battle()
```

```
=== Arthur vs Mordred ===
Hero:  Character Arthur: 100/100 HP, base damage 10, weapon: none
Enemy: Character Mordred: 100/100 HP, base damage 8, weapon: none
Round 1
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 92/100 HP | Mordred: 90/100 HP
Round 2
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 84/100 HP | Mordred: 80/100 HP
Round 3
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 76/100 HP | Mordred: 70/100 HP
Round 4
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 68/100 HP | Mordred: 60/100 HP
Round 5
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 60/100 HP | Mordred: 50/100 HP
Round 6
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 52/100 HP | Mordred: 40/100 HP
Round 7
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 44/100 HP | Mordred: 30/100 HP
Round 8
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 36/100 HP | Mordred: 20/100 HP
Round 9
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  Mordred attacks Arthur with bare hands (8 dmg) and deals 8 damage.
  -> Arthur: 28/100 HP | Mordred: 10/100 HP
Round 10
  Arthur attacks Mordred with bare hands (10 dmg) and deals 10 damage.
  -> Arthur: 28/100 HP | Mordred: 0/100 HP
*** Arthur wins after 10 rounds with 28/100 HP left! ***
```

Battle 2: Warrior vs Mage, no weapons

In [21]:
```
battle = Battle(Warrior("Conan"), Mage("Merlin"))
winner = battle.simulate_battle()
```

```
=== Conan vs Merlin ===
Hero:  Warrior Conan: 100/100 HP, base damage 10, weapon: none, armor 5
Enemy: Mage Merlin: 100/100 HP, base damage 10, weapon: none, 30/30 mana
Round 1
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 85/100 HP | Merlin: 90/100 HP, 20/30 mana
Round 2
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 70/100 HP | Merlin: 80/100 HP, 10/30 mana
Round 3
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 55/100 HP | Merlin: 70/100 HP, 0/30 mana
Round 4
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with bare hands (10 dmg) and deals 5 damage.
  -> Conan: 50/100 HP | Merlin: 60/100 HP, 5/30 mana
Round 5
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with bare hands (10 dmg) and deals 5 damage.
  -> Conan: 45/100 HP | Merlin: 50/100 HP, 10/30 mana
Round 6
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 30/100 HP | Merlin: 40/100 HP, 0/30 mana
Round 7
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with bare hands (10 dmg) and deals 5 damage.
  -> Conan: 25/100 HP | Merlin: 30/100 HP, 5/30 mana
Round 8
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with bare hands (10 dmg) and deals 5 damage.
  -> Conan: 20/100 HP | Merlin: 20/100 HP, 10/30 mana
Round 9
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 5/100 HP | Merlin: 10/100 HP, 0/30 mana
Round 10
  Conan attacks Merlin with bare hands (10 dmg) and deals 10 damage.
  -> Conan: 5/100 HP | Merlin: 0/100 HP, 0/30 mana
*** Conan wins after 10 rounds with 5/100 HP left! ***
```

Battle 3: Warrior vs Mage, with weapons

Now the warrior has a `Greatsword` (d20) and the mage has a `Staff` (d6) for the turns when it is out of mana.

In [22]:
```
battle = Battle(Warrior("Conan", weapon=greatsword), Mage("Merlin", weapon=staff))
winner = battle.simulate_battle()
```

```
=== Conan vs Merlin ===
Hero:  Warrior Conan: 100/100 HP, base damage 10, weapon: Greatsword (1d20), armor 5
Enemy: Mage Merlin: 100/100 HP, base damage 10, weapon: Staff (1d6), 30/30 mana
Round 1
  Conan attacks Merlin with Greatsword (1d20) and deals 19 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 85/100 HP | Merlin: 81/100 HP, 20/30 mana
Round 2
  Conan attacks Merlin with Greatsword (1d20) and deals 2 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 70/100 HP | Merlin: 79/100 HP, 10/30 mana
Round 3
  Conan attacks Merlin with Greatsword (1d20) and deals 18 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 55/100 HP | Merlin: 61/100 HP, 0/30 mana
Round 4
  Conan attacks Merlin with Greatsword (1d20) and deals 4 damage.
  Merlin attacks Conan with Staff (1d6) and deals 0 damage.
  -> Conan: 55/100 HP | Merlin: 57/100 HP, 5/30 mana
Round 5
  Conan attacks Merlin with Greatsword (1d20) and deals 5 damage.
  Merlin attacks Conan with Staff (1d6) and deals 0 damage.
  -> Conan: 55/100 HP | Merlin: 52/100 HP, 10/30 mana
Round 6
  Conan attacks Merlin with Greatsword (1d20) and deals 20 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 40/100 HP | Merlin: 32/100 HP, 0/30 mana
Round 7
  Conan attacks Merlin with Greatsword (1d20) and deals 6 damage.
  Merlin attacks Conan with Staff (1d6) and deals 1 damage.
  -> Conan: 39/100 HP | Merlin: 26/100 HP, 5/30 mana
Round 8
  Conan attacks Merlin with Greatsword (1d20) and deals 12 damage.
  Merlin attacks Conan with Staff (1d6) and deals 0 damage.
  -> Conan: 39/100 HP | Merlin: 14/100 HP, 10/30 mana
Round 9
  Conan attacks Merlin with Greatsword (1d20) and deals 8 damage.
  Merlin attacks Conan with a magic spell (20 dmg, 10 mana) and deals 15 damage.
  -> Conan: 24/100 HP | Merlin: 6/100 HP, 0/30 mana
Round 10
  Conan attacks Merlin with Greatsword (1d20) and deals 6 damage.
  -> Conan: 24/100 HP | Merlin: 0/100 HP, 0/30 mana
*** Conan wins after 10 rounds with 24/100 HP left! ***
```

Battle 4: Mage vs Mage, with weapons

In [23]:
```
battle = Battle(Mage("Merlin", weapon=staff), Mage("Morgana", weapon=dagger))
winner = battle.simulate_battle()
```

```
=== Merlin vs Morgana ===
Hero:  Mage Merlin: 100/100 HP, base damage 10, weapon: Staff (1d6), 30/30 mana
Enemy: Mage Morgana: 100/100 HP, base damage 10, weapon: Dagger (1d4), 30/30 mana
Round 1
  Merlin attacks Morgana with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  Morgana attacks Merlin with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  -> Merlin: 80/100 HP, 20/30 mana | Morgana: 80/100 HP, 20/30 mana
Round 2
  Merlin attacks Morgana with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  Morgana attacks Merlin with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  -> Merlin: 60/100 HP, 10/30 mana | Morgana: 60/100 HP, 10/30 mana
Round 3
  Merlin attacks Morgana with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  Morgana attacks Merlin with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  -> Merlin: 40/100 HP, 0/30 mana | Morgana: 40/100 HP, 0/30 mana
Round 4
  Merlin attacks Morgana with Staff (1d6) and deals 5 damage.
  Morgana attacks Merlin with Dagger (1d4) and deals 3 damage.
  -> Merlin: 37/100 HP, 5/30 mana | Morgana: 35/100 HP, 5/30 mana
Round 5
  Merlin attacks Morgana with Staff (1d6) and deals 4 damage.
  Morgana attacks Merlin with Dagger (1d4) and deals 2 damage.
  -> Merlin: 35/100 HP, 10/30 mana | Morgana: 31/100 HP, 10/30 mana
Round 6
  Merlin attacks Morgana with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  Morgana attacks Merlin with a magic spell (20 dmg, 10 mana) and deals 20 damage.
  -> Merlin: 15/100 HP, 0/30 mana | Morgana: 11/100 HP, 0/30 mana
Round 7
  Merlin attacks Morgana with Staff (1d6) and deals 6 damage.
  Morgana attacks Merlin with Dagger (1d4) and deals 3 damage.
  -> Merlin: 12/100 HP, 5/30 mana | Morgana: 5/100 HP, 5/30 mana
Round 8
  Merlin attacks Morgana with Staff (1d6) and deals 2 damage.
  Morgana attacks Merlin with Dagger (1d4) and deals 3 damage.
  -> Merlin: 9/100 HP, 10/30 mana | Morgana: 3/100 HP, 10/30 mana
Round 9
  Merlin attacks Morgana with a magic spell (20 dmg, 10 mana) and deals 3 damage.
  -> Merlin: 9/100 HP, 0/30 mana | Morgana: 0/100 HP, 10/30 mana
*** Merlin wins after 9 rounds with 9/100 HP left! ***
```

Battle 5: A battle nobody can win

Two warriors with 5 armor and only 4 base damage cannot hurt each other. Thanks to `max_rounds` the battle ends in a draw instead of looping forever (we use `max_rounds=5` to keep the output short).

In [24]:
```python
battle = Battle(Warrior("Ironclad", damage=4), Warrior("Stonewall", damage=4), max_rounds=5)
result = battle.simulate_battle()
print("simulate_battle() returned:", result)
```

```
=== Ironclad vs Stonewall ===
Hero:  Warrior Ironclad: 100/100 HP, base damage 4, weapon: none, armor 5
Enemy: Warrior Stonewall: 100/100 HP, base damage 4, weapon: none, armor 5
Round 1
  Ironclad attacks Stonewall with bare hands (4 dmg) and deals 0 damage.
  Stonewall attacks Ironclad with bare hands (4 dmg) and deals 0 damage.
  -> Ironclad: 100/100 HP | Stonewall: 100/100 HP
Round 2
  Ironclad attacks Stonewall with bare hands (4 dmg) and deals 0 damage.
  Stonewall attacks Ironclad with bare hands (4 dmg) and deals 0 damage.
  -> Ironclad: 100/100 HP | Stonewall: 100/100 HP
Round 3
  Ironclad attacks Stonewall with bare hands (4 dmg) and deals 0 damage.
  Stonewall attacks Ironclad with bare hands (4 dmg) and deals 0 damage.
  -> Ironclad: 100/100 HP | Stonewall: 100/100 HP
Round 4
  Ironclad attacks Stonewall with bare hands (4 dmg) and deals 0 damage.
  Stonewall attacks Ironclad with bare hands (4 dmg) and deals 0 damage.
  -> Ironclad: 100/100 HP | Stonewall: 100/100 HP
Round 5
  Ironclad attacks Stonewall with bare hands (4 dmg) and deals 0 damage.
  Stonewall attacks Ironclad with bare hands (4 dmg) and deals 0 damage.
  -> Ironclad: 100/100 HP | Stonewall: 100/100 HP
No winner after 5 rounds - it's a draw!
simulate_battle() returned: None
```

Many simulated battles

One battle with weapons says little, because the outcome depends on the dice. To compare the character types we simulate each match-up 10 000 times (with `verbose=False`) and count how often each side wins. New characters are created for every battle, since a dead character cannot fight again. The hero always attacks first in every round.

In [25]:

```python
def run_matchup(make_hero, make_enemy, n_battles: int = 10_000) -> dict:
    """Simulate n_battles between fresh characters and return win rates and average battle length."""
    results = Counter()
    total_rounds = 0
    for _ in range(n_battles):
        battle = Battle(make_hero(), make_enemy(), verbose=False)
        winner = battle.simulate_battle()
        results["draw" if winner is None else ("hero" if winner is battle.hero else "enemy")] += 1
        total_rounds += battle.rounds_played
    return {
        "hero": 100 * results["hero"] / n_battles,
        "enemy": 100 * results["enemy"] / n_battles,
        "draw": 100 * results["draw"] / n_battles,
        "rounds": total_rounds / n_battles,
    }


matchups = {
    "Character vs Character (no weapons)": (lambda: Character("Hero"), lambda: Character("Enemy")),
    "Character + Greatsword vs Character (no weapon)": (
        lambda: Character("Hero", weapon=greatsword), lambda: Character("Enemy")),
    "Character (no weapon) vs Character + Greatsword": (
        lambda: Character("Hero"), lambda: Character("Enemy", weapon=greatsword)),
    "Warrior vs Mage (no weapons)": (lambda: Warrior("Hero"), lambda: Mage("Enemy")),
    "Mage vs Warrior (no weapons)": (lambda: Mage("Hero"), lambda: Warrior("Enemy")),
    "Warrior + Greatsword vs Mage + Staff": (
```

```
        lambda: Warrior("Hero", weapon=greatsword), lambda: Mage("Enemy", weapon=staff)),
    "Mage + Staff vs Warrior + Greatsword": (
        lambda: Mage("Hero", weapon=staff), lambda: Warrior("Enemy", weapon=greatsword)),
    "Warrior + Dagger vs Mage + Staff": (
        lambda: Warrior("Hero", weapon=dagger), lambda: Mage("Enemy", weapon=staff)),
    "Warrior + Greatsword vs Warrior + Greatsword": (
        lambda: Warrior("Hero", weapon=greatsword), lambda: Warrior("Enemy", weapon=greatsword)),
    "Mage + Staff vs Mage + Staff": (
        lambda: Mage("Hero", weapon=staff), lambda: Mage("Enemy", weapon=staff)),
}

rows = ["| Match-up (hero vs enemy) | Hero wins | Enemy wins | Draws | Avg. rounds |",
        "|---|---:|---:|---:|---:|"]
for label, (make_hero, make_enemy) in matchups.items():
    r = run_matchup(make_hero, make_enemy)
    rows.append(f"| {label} | {r['hero']:.1f} % | {r['enemy']:.1f} % | {r['draw']:.1f} % |
{r['rounds']:.1f} |")
display(Markdown("\n".join(rows)))
```

Match-up (hero vs enemy)	Hero wins	Enemy wins	Draws	Avg. rounds
Character vs Character (no weapons)	100.0 %	0.0 %	0.0 %	10.0
Character + Greatsword vs Character (no weapon)	61.5 %	38.5 %	0.0 %	9.4
Character (no weapon) vs Character + Greatsword	60.9 %	39.1 %	0.0 %	9.4
Warrior vs Mage (no weapons)	100.0 %	0.0 %	0.0 %	10.0
Mage vs Warrior (no weapons)	100.0 %	0.0 %	0.0 %	10.0
Warrior + Greatsword vs Mage + Staff	99.5 %	0.5 %	0.0 %	10.1
Mage + Staff vs Warrior + Greatsword	1.4 %	98.6 %	0.0 %	10.1
Warrior + Dagger vs Mage + Staff	0.0 %	100.0 %	0.0 %	15.0
Warrior + Greatsword vs Warrior + Greatsword	54.0 %	46.0 %	0.0 %	15.5
Mage + Staff vs Mage + Staff	95.0 %	5.0 %	0.0 %	8.9

Discussion of results

1. Attacking first is a big advantage. In a battle where both sides need the same number of hits, the hero always wins, because the enemy never gets its last attack. Without weapons there is no randomness, so *Character vs Character* and *Warrior vs Mage* are won by the hero 100 % of the time. Warrior vs Mage without weapons is in fact perfectly balanced: both need exactly 10 rounds to kill the other, so whoever goes first wins (this also happens in *Mage vs Warrior*). Even when the dice are involved, going first helps: two identical warriors with greatswords give the hero about 54 % wins, and two identical mages give the hero about 95 % wins. The mage match-up is so lopsided because most of a mage's damage comes from spells, which always deal exactly 20 damage - so the mage that casts first also finishes first.

2. A weapon is not always better than bare hands. In our rules a weapon *replaces* the base damage of 10. A die with *n* sides gives on average (n+1)/2 damage, so only the `Greatsword` (d20, mean 10.5) is on average better than bare hands, and only slightly. This is why the hero wins about 61 % in both *Character + Greatsword vs Character* and *Character vs Character + Greatsword*: the greatsword does not make up for attacking second. A weak weapon is a disadvantage: a `Warrior` with a `Dagger` (d4) loses every single time against a `Mage` with a `Staff`.

3. Armor is very strong against small dice. With 5 armor, a `Staff` (d6) only gets through on a roll of 6, and then deals just 1 damage. So when a mage runs out of mana and attacks a warrior with a staff, it deals almost no damage, and the warrior with a greatsword wins about 99 % of the battles no matter who starts. Funnily enough, the mage would do better against a warrior *without* the staff, since bare hands deal 10 - 5 = 5 damage.

4. Randomness makes the battles longer and less predictable. The battles without weapons always take exactly 10 rounds, while the battles with dice vary. Between two warriors with greatswords the average battle takes about 15.5 rounds, since armor blocks more of the low rolls.

Possible improvements. To make the game fairer, the starting character could be chosen by a dice roll (initiative) instead of always letting the hero start. It would also make sense to balance weapon damage against base damage, for example by letting a weapon *add* to the base damage instead of replacing it.

Testing

All classes are tested in `test_game.py` with `pytest`. To make tests with weapons predictable, the tests use a small `FixedDice` subclass of `Dice` whose `roll()` always returns the same number. The tests check, among other things, that HP stays between 0 and `hp_max`, that `attack()` goes through the target's `take_damage()`, that armor and spells work as described, the mage's spell/attack pattern, and that a battle always ends with exactly one survivor (or a draw when no damage is possible).

In [26]:
```
import subprocess
import sys

result = subprocess.run([sys.executable, "-m", "pytest", "-q", "--color=no", "test_game.py"],
capture_output=True, text=True)
print((result.stdout + result.stderr)[-400:])
```

```
.........................................                                [100%]
44 passed in 0.05s
```

Use of generative AI

See `GENAI_USE.md` for our discussion of how we used generative AI in this deliverable.